

## Continuation of Yesterday's Work

### 5. Props (Properties)

- **What are Props?** Props represent **input data passed into components** to make them dynamic and customizable.
- **How They Work:** You pass props as key-value attributes when rendering a component (e.g., `<Card name="Priyanka" />`).
- **Accessing Props:** They are received as an object in the component's first parameter, which can be destructured or accessed directly via **props.name** inside curly braces `{ }` in your JSX.

### 6. State Management (useState Hook)

- **What is State?** A **state is a local variable** in a component that stores data.
- **Dynamic UI & Rerendering:** When a state variable's value changes, **React automatically updates and rerenders the UI** to reflect the new data.
- **useState Hook:** Inside a functional component, state is declared using the **useState** hook. Calling **useState** returns an array containing:
  1. The **state variable** (holding the current value).
  2. A **setter function** (used to update the state).
- **Event Handling:** React handles events like button clicks using **camelCase** attributes (e.g., **onClick**) and takes a **function reference** inside curly braces instead of an inline string call.

### 7. Context API

- **What is Context API?** It is a technique used to **share data globally across components**.
- **Solving "Props Drilling":** It resolves the issue where data has to be passed down manually through multiple nested layers of parent-child components (from Component A down to E) just to reach a deeply nested child.
- **Three Key Steps:**
  1. **Create Context:** Use the **createContext()** function to create a context reference.
  2. **Provide Context:** Use `<MyContext.Provider value={state}>` in the parent component to wrap child components and supply the data.
  3. **Consume Context:** Use the **useContext(MyContext)** hook inside any child component to directly access the shared data.

### 8. React Routing (react-router-dom)

- **Purpose:** Allows developers to implement routing so users can switch from one view to another in a **Single Page Application (SPA) without reloading the page**.
- **Configuration:**
  - Wrap your routes inside a **<Routes>** component.
  - Map individual paths to component elements using `<Route path="/profile" element={<Profile />} />`.
- **Reload-Free Navigation:** Replace standard HTML `<a>` tags with React Router's **<Link>** component and the **href** attribute with the **to** prop (e.g., `<Link to="/profile">`) to achieve fast, reload-free navigation.

## Solved 25 MCQs on Reactjs

MCQ Link: <https://www.interviewbit.com/react-mcq/>

### Notes:

#### 1. `setState()` inside `render()`

- Calling `setState()` inside the `render()` method causes the component to **re-render continuously**.
- This creates an **infinite loop**, eventually resulting in a **Stack Overflow error**.
- `setState()` should not be called inside `render()`.

#### 2. React Covers the View Layer

- React.js mainly covers the **view layer** of an application.
- It is primarily used to create and manage the **User Interface (UI)**.

#### 3. `render()` Method

- The `render()` method is used to render HTML/UI to a web page in React.
- In modern functional components, JSX returned from the component serves this purpose.

#### 4. Variables in ES6

There are **three ways** of defining variables:

1. `var`
2. `let`
3. `const`

#### 5. `preventDefault()`

- `preventDefault()` is used to prevent the **default behavior of an event**.
- Example: Preventing a form from submitting automatically.

`event.preventDefault();`

#### 6. Props

- **Props** are arbitrary inputs passed to React components.
- They allow components to receive **data from their parent component**.
- Props are **read-only**.

Example:

```
<Card name="Rajiv Shukla" />
```

#### 7. `disabled={false}`

- If an input element has **disabled={false}**, the input will be **enabled**.
- **false** means the **disabled** property is not applied.

## 8. Babel

- **Babel** is a **JavaScript compiler**.
- It converts modern JavaScript/JSX code into JavaScript that can be understood by browsers.

## 9. ReactJS - Main Use

- ReactJS is mainly used for building **User Interfaces (UIs)**.
- It is especially useful for creating **dynamic and interactive web applications**.

## 10. What is ReactJS?

- ReactJS is a **JavaScript library for building user interfaces**.
- It was originally developed at Facebook.

## 11. Webpack

- **Webpack** is a **module bundler**.
- It takes different files/modules and bundles them together so they can be efficiently used by a web application.

## 12. Code Splitting — `React.lazy`

- `React.lazy` is used for **code splitting** in React.
- It allows components to be loaded only when they are needed.

Example:

```
const Profile = React.lazy(() => import("./Profile"));
```

## 13. `useReducer` vs `useState`

- `useState` is generally used for simple state management.
- `useReducer` is useful when state logic is **more complex** or involves multiple related state updates.
- `useReducer` is not simply a replacement for Redux.

## 14. `setState()` and State Objects

- In **class components**, `setState()` updates the component's state.
- When an object is provided to `setState()`, React merges it with the current state.

## 15. Who Developed ReactJS?

- ReactJS was developed by **Facebook** (now Meta).
- It was initially released as an open-source project by Facebook.

## 16. Two Ways to Handle Data in React

The two important ways of handling data in React are:

1. **Props**
2. **State**
  - **Props:** Data passed from parent to child.
  - **State:** Data managed inside a component.

## 17. Changing State in Class Components

- In React **class components**, **this.setState()** is used to change/update state.

Example:

```
this.setState({ count: 1 });
```

## 18. Virtual DOM

- React uses a **Virtual DOM** to improve performance.
- React compares changes in the Virtual DOM and updates only the necessary parts of the actual DOM.

## 19. Calling a Function in JSX

- A function can be called inside JSX using **curly braces {}**.

Example:

```
<h1>{fetch()}</h1>
```

- The value returned by `fetch()` will be rendered inside the `<h1>` element.

## 20. Accessing State in Class Components

- In a React class component, state is accessed using:

```
this.state
```

- The given answer **this.values** is incorrect for accessing React state.

Example:

```
this.state.name
```

## 21. Webpack Dev Server Default Port

- The default port commonly associated with **webpack-dev-server** is:

8080

## 22. React Applications are Declarative

- React applications are commonly described as **declarative**.

- In declarative programming, you describe **what the UI should look like**, rather than manually specifying every DOM operation.

## 23. Creator of React.js

- **Jordan Walke** created React.js.
- He was a software engineer at Facebook.

## 24. ref in React

- A **ref** is used to directly access a **DOM node** or a React element.
- It can be useful when you need to interact with the DOM directly.

Example:

```
<input ref={inputRef} />
```

## 25. useEffect

- **useLayoutEffect** runs after React has updated the DOM but **before the browser paints the screen**.
- It is useful when you need to **measure or modify the DOM before the user sees the result**.
- This differs from **useEffect**, which generally allows the browser to paint before the effect runs.